AnyBlox: Decoupling Storage Formats from Data Processing Systems

Liam Sanft
liam.sanft@mailbox.tu-dresden.de
TU Dresden
Germany, Dresden

Abstract

Open storage formats in data lakes have separated data from the systems that process it, reducing vendor lock-in. However, reading a format still requires a decoder for that format to be present in every consuming system. For N systems and M formats, this results in $N \times M$ independent decoder implementations, which raise maintenance costs, increase the risk of security vulnerabilities, and discourage the adoption of new encodings. To address this, Gienieczko et al. (2025) propose AnyBlox, a framework that moves the decoder into the data itself: each dataset ships with its own decoder as sandboxed WebAssembly bytecode. With this approach, any engine that implements the framework once can read a format without prior knowledge, reducing the integration burden to $N + M$. The sandbox overhead remains negligible in end-to-end workloads. Furthermore, more efficient encodings outperform established formats in both scan latency and compression, directly addressing the maintenance burden and format ossification caused by the $N \times M$ problem.

1 Introduction

For decades, database systems functioned as tightly coupled units, with a single system controlling storage format, processing logic, and data access. External systems submitted queries and received results; internal operations remained hidden. With complete control over the physical layout, systems can optimise data storage and processing within their own boundaries. This model was sufficient as long as data did not cross system boundaries [19].

However, the volume and variety of collected data have grown drastically. Data now arrives continuously from a growing number of sources, in formats suited to their respective domains rather than to any particular database system. As a result, the database landscape has shifted from a single system controlling all aspects of data management to a heterogeneous, modular ecosystem of specialised systems, each handling a distinct part of the data pipeline using a shared storage layer. Open file formats such as Apache Parquet[1] and Apache ORC[2], stored in data lakes [3], are widely adopted in this context. Beyond these, data scientists increasingly analyse data using domain-specific encodings from fields such as high-energy physics and bioinformatics [8].

Independently, research in data compression and encoding continues to produce formats that surpass established standards for storage efficiency and query throughput. Yet, these advances rarely appear in production systems, because each system must independently implement support for every new format it wants to read [8].

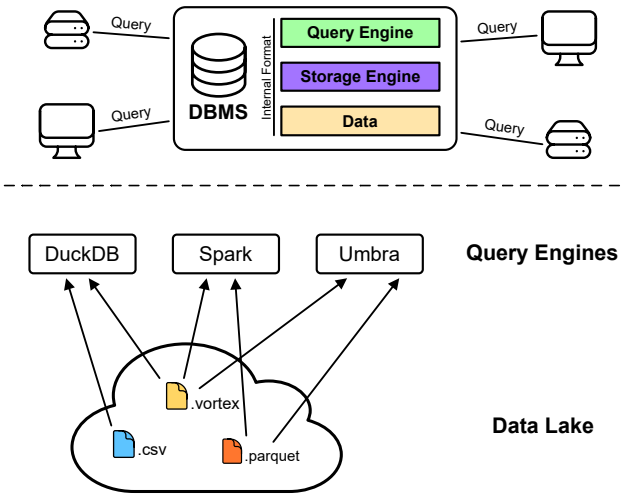

Figure 1: (Top) A monolithic DBMS controls its query engine, storage engine, and internal format; clients interact only through queries. (Bottom) In a data lake, multiple query engines access a shared pool of open file formats. Since each must implement a decoder for each format, N engines and M formats require $N \times M$ independent implementations.

Both trends lead to the same structural problem. Codd's principle of physical data independence [6] assumes a system controls its storage format and shields applications from physical details. In a shared storage layer, this does not apply. The data producer determines the format, so any system that reads a format must implement its own decoder. With N systems and M formats, there are $N \times M$ independent implementations to develop and maintain.

This results in higher implementation costs, greater maintenance effort, and increased security risks, as each implementation may contain bugs. For example, a high-severity vulnerability in PostgreSQL's[3] JSON support went undetected for over five years [17]. This initial hurdle discourages systems from supporting new formats altogether.

The consequence is a self-sustaining cycle: systems do not adopt a new file format because its user base is too small, and the user base remains small because the format lacks support. Gienieczko et al. (2025) identify this as a core structural problem because an encoding must be popular enough to justify support, but cannot gain popularity without broad support.

[1] Apache Parquet, https://parquet.apache.org (accessed: May 22, 2026)
[2] Apache ORC, https://orc.apache.org (accessed: May 22, 2026)
[3] PostgreSQL, https://www.postgresql.org (accessed: May 22, 2026)

This dynamic leads to format ossification, where datasets use outdated formats just to stay compatible, even when better encodings exist [11]. Changing formats is difficult because it risks breaking compatibility with many systems that use older specifications.

But what could solve this combinatorial integration burden? An abstraction layer between storage formats and processing systems reduces the $N \times M$ problem to $N + M$. Each system implements the interface once, and each format provides a single implementation against it. Gienieczko et al. (2025) propose exactly this and present AnyBlox as their realisation.

2 Self-Decoding Data: The AnyBlox Design

A future-proof data access solution must simultaneously satisfy two conceptual requirements. First, it must introduce an abstraction layer that decouples format internals from processing systems and system internals from decoders, reducing the $N \times M$ integration burden to $N + M$. Second, this abstraction cannot preclude direct file access, since routing data through an intermediary process introduces transfer overhead that can dominate total runtime. These two goals are in tension: an abstraction layer must reside somewhere, and each possible location either breaks direct access or reintroduces format-specific coupling [8].

Existing approaches each resolve this tension in favour of one side. Native integration preserves direct file access and achieves the best performance through tight coupling with host internals, but requires a separate decoder implementation for each format and system, thereby reproducing the $N \times M$ problem. Extension mechanisms reduce the per-format implementation effort, but each host exposes its own extension API, so a decoder written for system A cannot be reused in system B without a near-complete rewrite. Isolated extensions, such as containerised user-defined functions [18] or remote cloud user-defined code execution solutions, provide both isolation and portability, but introduce a data transfer bottleneck between the host and the isolated decoder process [8].

None of these approaches satisfies both requirements at once. Native integration, extensions, and isolated extensions each forgo one of them, while static verification targets both but is held back by the limitations of current verifier technology [8]. Gienieczko et al. (2025) therefore argue that only one location for the decoder remains: inside the data file itself. Packaging the decoder with the data eliminates the transfer bottleneck, since decoder and data reside in the same address space, while the abstraction is preserved because the host need not implement any format-specific logic. This is the core idea behind self-decoding data and the design premise of AnyBlox.

2.1 WebAssembly as the Decoder Runtime

A self-decoding dataset requires a decoder runtime that satisfies four criteria: **Portability**, **Security**, **Performance** and **Extensibility**.

Building on these criteria, Gienieczko et al. (2025) identify WebAssembly (Wasm) as a runtime that satisfies all four simultaneously. Wasm is a portable low-level bytecode and virtual machine specification, designed as an abstraction over modern hardware rather than a commitment to any specific programming model, making it language-, hardware-, and platform-independent [9].

2.1.1 Portability. A decoder implemented in Wasm can operate across a broad set of host architectures, including browsers, mobile devices, embedded systems, and server workstations. With AnyBlox, a format author can write and compile a decoder once and execute it on any host through `libanyblox`, regardless of the underlying hardware or operating system [8].

2.1.2 Security. The WebAssembly specification provides machine-verifiable guarantees of memory safety and isolation [21]. Building on this foundation, AnyBlox reinforces these guarantees by being implemented predominantly in safe Rust, ensuring memory and thread safety. Furthermore, all unsafe code is confined to a single module that handles Wasm memory maps, making it straightforward to inspect [8].

2.1.3 Performance. Wasm is JIT-compiled to the host's native instruction set at runtime, enabling near-native performance. Additionally, AnyBlox avoids unnecessary data copying through a custom memory management scheme, which improves efficiency. As observed by Gienieczko et al. (2025), AnyBlox further benefits by passively inheriting advances in Wasm compiler technology, since all JIT and sandboxing details are encapsulated within `libanyblox` and not exposed in the public API [8].

2.1.4 Extensibility. Finally, in terms of extensibility, since most general-purpose programming languages provide a Wasm toolchain, format authors can implement decoders in their language of choice. The main practical obstacle is SIMD instructions: converting platform-specific SIMD intrinsics to Wasm's instruction set requires manual developer effort. However, since Wasm defines SIMD at the bytecode level, the resulting modules remain fully portable across architectures [8].

2.2 Reading a Dataset: From File to Query Pipeline

To read an AnyBlox dataset, the Host opens the .any file and reads its metadata during query planning. The metadata specifies the schema, row count, an estimate of the decoded data size, and recommended batch size, which are used to partition workloads across threads and to schedule scans before decoding.

The Host passes metadata and the .any file to libanyblox, which extracts, JIT-compiles, and caches the Wasm Decoder module. Later uses of the same Decoder skip compilation. AnyBlox then sets up the Decoder's execution environment and returns a job handle to the Host.

This procedure describes two distinct AnyBlox modes. In standalone mode, the dataset resides in its own `.any` file with a thin metadata layer. AnyBlox reliably encodes the schema and row count; further hints, such as an estimated decoded size or a minimum recommended batch size, may be present but are treated as optional. In embedded mode, AnyBlox is incorporated within an existing host format (such as Parquet). Here, the Host uses its own decoding library, and the embedded path inherits the Host format's statistics, column-level filtering and row-group filtering. The decoding pipeline described below is identical in both modes. The key difference is the metadata available to the Host, which is relevant for predicate pushdown (Section 3.3).

From this point, the Host repeatedly requests arbitrary tuple ranges from that handle. Modern query engines process data in batches to reduce communication overhead between operators. Therefore, rather than decoding the entire dataset at once, the Host issues successive calls for ranges of tuples [14]. AnyBlox delegates each such request into the sandboxed Wasm module and returns the decoded result to the Host. The sole function through which this happens is `decode_batch` [8]:

- `i32 data`: pointer to the start of the encoded data in Wasm linear memory
- `i32 data_length`: length of the encoded data in bytes
- `i32 start_tuple`: ID of the first tuple to decode
- `i32 tuple_count`: number of tuples to decode
- `i32 state`: pointer to the state page in Wasm linear memory
- `i64 projection_mask`: bitmask specifying which columns to decode, allowing the Host to skip columns that are not needed for the current query

The `data` parameter references the encoded dataset in the Decoder's execution environment. Copying the dataset into Decoder-accessible memory incurs an initialisation cost proportional to the dataset size. AnyBlox instead uses `mmap` to map the dataset file into the Decoder's linear memory, avoiding copying. Physical memory is committed only upon first page access.

Wasm's linear memory is a contiguous, byte-addressable memory region starting at address `0x0` that represents the entire address space accessible to the Decoder, isolated from the rest of the host process. AnyBlox controls its layout. The first pages hold the Decoder module's code, stack, and static data (typically a few megabytes). The encoded dataset is mapped via `mmap` into the first page after this initial region, with the state page directly following the dataset; Decoder allocations expand from the following pages. The `data` parameter thus points to the start of the mapped dataset. Reusing this linear memory region across jobs on the same thread eliminates the need for repeated initialisation.

The `state` parameter points to a page within the linear memory that is zero-initialised at the start of every job. A Decoder can use it to cache metadata or memory allocations across successive `decode_batch` calls. For example, a run-end encoding Decoder that performs a binary search on the first call can store its current position on the state page and resume directly on the next call, skipping the search entirely. Because the state page resets at the start of each job, Decoders are idempotent: identical parameters applied to the same dataset always produce the same results.

Once decoding is complete, `decode_batch` returns a pointer to an Apache Arrow Record Batch within the linear memory. Arrow[4] is a columnar in-memory format developed by the Apache Software Foundation. Rather than storing data row by row, it organises each column as a contiguous array in memory, allowing query engines to efficiently process large amounts of data without unnecessary memory accesses. A Record Batch is a collection of such arrays, all of the same length, representing a horizontal slice of a table. Arrow standardises the representation of common SQL types (e.g., integers, floating-point numbers, strings, dates, and more). Arrow serves as a uniform intermediate representation: instead of pairing each of the N Decoders with each of the M Hosts, both sides target Arrow, reducing the number of required translations from $N \times M$ to $N + M$. A Decoder therefore needs no knowledge of the specific Host it feeds, and vice versa. Every Decoder, regardless of the underlying encoding scheme, produces an Arrow Record Batch, and every Host that speaks Arrow can consume it without any AnyBlox-specific logic. AnyBlox translates the Wasm-internal pointer into a native address and passes the batch to the Host, where it enters the query pipeline. Integration depth varies by system: DataFusion [12] uses Arrow as its internal representation, allowing zero-copy transfers, while DuckDB [16] and Spark[5] convert Arrow batches into their own formats on the fly. In all three cases this relies on existing Arrow support, with no AnyBlox-specific code [8].

3 Discussion

The preceding sections introduced the AnyBlox architecture and its decoding model. Whether this design is practical depends on three questions:

(1) What overhead does the Wasm sandbox introduce?
(2) How much effort does integrating the framework into an engine require?
(3) Where does the approach reach its limits?

To address these questions, Gienieczko et al. (2025) provide both benchmarks and integration experience covering favourable and unfavourable cases for AnyBlox.

3.1 Performance Evaluation

Sandboxing a decoder inside Wasm adds an indirection layer between the data and the query engine. Gienieczko et al. (2025) measure the resulting overhead directly. They run TPC-H at scale factor 20 in DataFusion [12] and vary the representation of `lineitem`, the largest table in the benchmark and therefore the one whose scan dominates the runtime, across three configurations:

Parquet read by DataFusion's native reader with Snappy[6] compression;
AnyBlox Vortex a Vortex file decoded inside the AnyBlox sandbox;
Native Vortex the same Vortex decoder compiled natively without any sandbox.

The first variant isolates the effect of the format, while the latter two isolate the effect of the sandbox. Predicate pushdown was disabled throughout, so the numbers reflect only scan and operator costs. Vortex[7] is a recent columnar format.

The benchmark yields two results, summarised in Figure 2. Both Vortex variants achieve over 2× lower scan latency than Parquet while also reaching better compression rates. For the Parquet reader, decoding accounts for nearly half of the processing time, while Vortex spends around 80% of its processing time in relational operators. The overhead of the sandbox itself is small. Sandboxed Vortex decoding adds 5% to the scan latency compared to native decoding, resulting in a 1% increase in overall query latency, which the authors report as within measurement noise. Across this end-to-end

[4] Apache Arrow, https://arrow.apache.org (accessed: June 2, 2026)

[5] Apache Spark, https://spark.apache.org (accessed: June 2, 2026)
[6] Snappy, https://github.com/google/snappy (accessed: June 15, 2026)
[7] Vortex, https://vortex.dev (accessed: June 15, 2026)

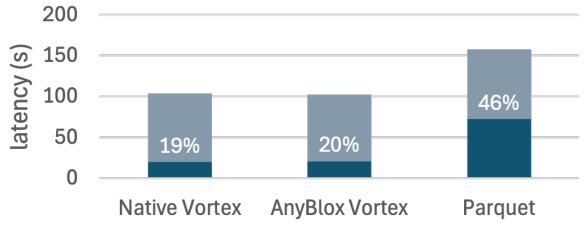

Figure 2: Total query latency (median, summed over 16 TPC-H queries at scale factor 20) in DataFusion, split into scan time (dark) and operator time (light). Percentages denote the scan share of total latency. Data source: [8].

workload, the cost of the sandbox is therefore negligible, and AnyBlox provides DataFusion with an encoding that outperforms its own native Parquet reader [8].

The microbenchmarks show where the sandbox's overhead appears. FSST [5] is a lightweight scheme for compressing strings. Gienieczko et al. (2025) benchmark decoding three FSST-compressed string columns from the Taxpayer dataset, which is part of the Public BI Benchmark [7]. Decoded, the 1.7 GB AnyBlox file expands into 2.5 GB of Arrow batches in memory. In this isolated scenario, the Wasm Decoder achieves over 43% lower decode throughput than the same decoder compiled natively. The cause is architectural rather than incidental: the FSST decoder relies heavily on 8-byte reads, but Wasm uses a 32-bit address space by default, so each 8-byte read is emulated with two 4-byte reads. The authors describe this as a fundamental limitation of the current design: a 64-bit address space would remove the emulation, but its incompatibility with the SFI mechanism discussed in Section 3.3 rules it out [8].

Taken together, the benchmarks put the sandbox cost in perspective, though the strength of evidence differs across cases. For a complex decoder such as Vortex, overhead is measured directly: in the end-to-end DataFusion workload, it stays within measurement noise. Similarly, the more efficient encoding lets AnyBlox outperform the native Parquet reader. In FSST's worst case, the slowdown is clearly evident in the microbenchmark, though its effect on full queries is not directly measured. Because the scan accounts for only about a fifth of processing time in DataFusion, a decoding slowdown can affect only that share of the runtime. Thus, the microbenchmark isolates the decoder under its least favourable conditions.

3.2 Integration Effort

The second claim AnyBlox makes is portability: an engine can support the framework without a large implementation effort. For example, Gienieczko et al. (2025) integrated AnyBlox into four data processing systems using different execution paradigms: Umbra [15], DuckDB, DataFusion, and Spark. Notably, a single programmer with no prior experience in any host codebase performed all integrations. The authors report that no integration required more than 2,000 lines of code; Table 1 gives the per-system breakdown in lines of code and person-days. Each integration only added a new scan operator and required no further changes to the host engine, leaving its optimiser and execution layer untouched [8].

Table 1: Effort of integrating AnyBlox into each host system. The Spark integration additionally required a 433-line JNI bridge, reusable for any JVM host, which is not included in the counts below. Data source: [8].

System	Lines of code	Person-days
DataFusion	461	2
DuckDB	586	3
Spark	756	15
Umbra	1305	10

The authors caution that lines of code and integration time are not rigorous measures of software complexity. These figures indicate, rather than prove, that the framework ports easily. With that caveat, they are the empirical counterpart to the $N \times M$ argument: a one-time integration lets an engine read any format for which a decoder is provided [8].

3.3 Limitations

Two of AnyBlox's limitations stem from its design rather than its current implementation. The first is predicate pushdown, a technique that evaluates filter conditions during the scan phase. Rows that fail a condition are discarded before decoding and before reaching operators such as aggregations or joins. Decoding is computationally expensive, so prior filtering avoids processing data that would otherwise be discarded [19].

Standalone AnyBlox lacks a predicate pushdown API for row filtering and supports only projection pushdown, limiting scans to requested columns rather than rows. This limitation results from a design choice: row filtering in a format-agnostic framework requires a system- and format-independent predicate expression, which AnyBlox does not provide. This approach follows the philosophy that statistics and indexes should be decoupled from storage format and instead managed by the query engine or a dedicated acceleration layer. Consequently, the authors point to Substrait[8], a cross-language serialisation format for relational algebra, as a sign of industry interest in a standardised, system-independent representation of relational operations [8]. Whether such a format could supply the predicate expression AnyBlox lacks remains open.

This limitation applies only to standalone use. If AnyBlox is embedded into a host format like Parquet at the row-group or column level, the host's row-group and column filtering continues to apply, so the embedded path retains the host's pushdown logic.

The second limitation concerns memory. Standalone AnyBlox maps the entire dataset into the Wasm linear memory using `mmap` before decoding. The current `wasm32` implementation provides a 32-bit address space, so each mapped file is limited to 4 GB. The authors note that typical analytical deployments keep file sizes well below 4 GiB to allow inexpensive updates, so the restriction may have limited impact in practice. A 64-bit address space (WebAssembly

[8] Substrait, https://substrait.io (accessed: June 15, 2026)

Memory64[9]), standardised in late 2024, would lift the 4 GB limit in principle, but is not a drop-in fix. The SFI mechanism on which AnyBlox's security rests assumes a decoder can address at most 8 GiB; a 64-bit address space breaks that assumption, so adopting it would forfeit the current security guarantee. Its performance impact has not yet been established. Using `mmap` also prevents pushing column projections to remote files, because the entire dataset must reside in memory before the decoder is invoked. This is a relevant constraint in data lake deployments, where data resides on object stores. Partial remote reads via page-fault handling are outlined as future work but are not yet implemented [8].

4 Conclusion

Reading an unfamiliar storage format normally requires a reader written for that format inside every system that wants to consume it. AnyBlox removes this requirement by moving the reader into the data. A decoder travels with the dataset as sandboxed Wasm bytecode, and any engine that implements a single scan interface can decode the format without knowing it in advance. The cost of this design is the sandbox overhead, which the measurements of Gienieczko et al. (2025) show to be negligible for the end-to-end workloads tested.

The authors frame the underlying problem as an $N \times M$ problem: N systems each supporting M formats incur $N \times M$ implementation and maintenance effort [8]. An abstraction layer between the two sides reduces this to $N + M$, since each engine implements the interface once and each format provides one decoder. The authors draw this analogy explicitly from compiler construction, citing the language-independent intermediate representation of LLVM [13], which decouples M language frontends from N target backends through a shared representation. AnyBlox applies the same decoupling between storage formats and query engines. This mapping is suggestive rather than demonstrated: it explains why the design should work, but the parallel to compilers does not by itself show that formats and engines separate as cleanly as languages and target backends.

This decoupling exemplifies a broader shift toward composable database systems, replacing large, monolithic designs with modular layers. AnyBlox demonstrates this by contributing the storage-format layer and stands alongside efforts modularising other layers: composable-query engines [12], extensible execution-plan frameworks [4, 10], portable query optimizers [1, 2], and system-independent persistence [20]. AnyBlox's choice not to define a row-filtering predicate format (see Section 3.3) leaves a gap that Substrait could fill. These modular components are complementary, each standardising a specific interface between otherwise independent systems [8].

The authors cite LLVM as a precedent for successful modular frameworks, showing how shared, open representations can transform systems [8, 13]. AnyBlox applies this principle to storage formats, and its evaluation demonstrates that a format-agnostic decoding layer can be built with low overhead and integrated into various engines with modest effort. Although the LLVM analogy is plausible, the evidence rests on a single evaluation from the authors. Ultimately, whether modular stacks displace integrated systems requires broader adoption and independent validation. The paper's main result is that the decoding layer is no longer the primary obstacle.

[9]WebAssembly Memory64, https://github.com/WebAssembly/memory64 (accessed: June 15, 2026)

References

[1] Rana Alotaibi, Yuanyuan Tian, Stefan Grafberger, Jesús Camacho-Rodríguez, Nicolas Bruno, Brian Kroth, Sergiy Matusevych, Ashvin Agrawal, Mahesh Behera, Ashit Gosalia, Cesar Galindo-Legaria, Milind Joshi, Milan Potocnik, Beysim Sezgin, Xiaoyu Li, and Carlo Curino. 2024. Towards Query Optimizer as a Service (QOaaS) in a Unified LakeHouse Ecosystem: Can One QO Rule Them All? https://doi.org/10.48550/arXiv.2411.13704 arXiv:2411.13704 [cs.DB]

[2] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Prithviraj Pandian, Nikolay Laptev, and Ryan Marcus. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proceedings of the VLDB Endowment* 16, 12 (Aug. 2023), 3515–3527. https://doi.org/10.14778/3611540.3611544

[3] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics. (2021).

[4] Maximilian Bandle and Jana Giceva. 2021. Database Technology for the Masses: Sub-Operators as First-Class Entities. *Proceedings of the VLDB Endowment* 14, 11 (July 2021), 2483–2490. https://doi.org/10.14778/3476249.3476296

[5] Peter Boncz, Thomas Neumann, and Viktor Leis. 2020. FSST: Fast Random Access String Compression. *Proceedings of the VLDB Endowment* 13, 12 (Aug. 2020), 2649–2661. https://doi.org/10.14778/3407790.3407851

[6] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (June 1970), 377–387. https://doi.org/10.1145/362384.362685

[7] Bogdan Ghit, Diego Tomé, and Peter Boncz. [n. d.]. White-Box Compression: Learning and Exploiting Compact Table Representations. ([n. d.]).

[8] Mateusz Gienieczko, Maximilian Kuschewski, Thomas Neumann, Viktor Leis, and Jana Giceva. 2025. AnyBlox: A Framework for Self-Decoding Datasets. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4017–4031. https://doi.org/10.14778/3749646.3749672

[9] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web up to Speed with WebAssembly. *SIGPLAN Not.* 52, 6 (June 2017), 185–200. https://doi.org/10.1145/3140587.3062363

[10] Michael Jungmair and Jana Giceva. 2023. Declarative Sub-Operators for Universal Data Processing. *Proceedings of the VLDB Endowment* 16, 11 (July 2023), 3461–3474. https://doi.org/10.14778/3611479.3611539

[11] Laurens Kuiper. 2025. Query Engines: Gatekeepers of the Parquet File Format. https://duckdb.org/2025/01/22/parquet-encodings.html.

[12] Andrew Lamb, Yijie Shen, Daniël Heres, Jayjeet Chakraborty, Mehmet Ozan Kabak, Liang-Chi Hsieh, and Chao Sun. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *Companion of the 2024 International Conference on Management of Data (SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 5–17. https://doi.org/10.1145/3626246.3653368

[13] C. Lattner and V. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* IEEE, San Jose, CA, USA, 75–86. https://doi.org/10.1109/CGO.2004.1281665

[14] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-Driven Parallelism: A NUMA-aware Query Evaluation Framework for the Many-Core Age. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. ACM, Snowbird Utah USA, 743–754. https://doi.org/10.1145/2588555.2610507

[15] Thomas Neumann and Michael Freitag. [n. d.]. Umbra: A Disk-Based System with In-Memory Performance. ([n. d.]).

[16] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. https://doi.org/10.1145/3299869.3320212

[17] Red Hat, Inc. 2017. CVE-2017-15098. https://nvd.nist.gov/vuln/detail/CVE-2017-15098.

[18] Karla Saur, Tara Mirmira, Konstantinos Karanasos, and Jesús Camacho-Rodríguez. 2022. Containerized Execution of UDFs: An Experimental Evaluation. *Proceedings of the VLDB Endowment* 15, 11 (July 2022), 3158–3171. https://doi.org/10.14778/3551793.3551860

[19] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (seventh edition ed.). McGraw-Hill, New York, NY.

[20] Emil Tsalapatis, Ryan Hancock, Rakeeb Hossain, and Ali José Mashtizadeh. 2024. MemSnap μCheckpoints: A Data Single Level Store for Fearless Persistence. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, La Jolla CA

USA, 622–638. https://doi.org/10.1145/3620666.3651334

[21] Conrad Watt. 2018. Mechanising and Verifying the WebAssembly Specification. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2018)*. Association for Computing Machinery, New York, NY, USA, 53–65. https://doi.org/10.1145/3167082